

Which labels an AI actually uses on its own work

PIN-CHECK SKILL · JAKE TILLER · AUGUST 2026

The problem

A lot of AI tooling works like this. The model produces an answer, then attaches a label to its own answer saying how good it is. Verified or unverified. Pass, fail, or needs review. High confidence or low. Added nothing.

The label is supposed to let you audit the output without reading all of it. That only works if the model actually uses the labels. So I counted which ones it uses.

The test

I gave a model problems that answer themselves. One was a deploy script that fails on Windows because it hardcodes forward slashes. The cause is stated in the problem. There is nothing left to discover.

Then I asked it to analyse the problem and label each piece of its own analysis, including with a label meaning *this added nothing*.

On a problem that contains its own answer, that label is correct for every piece. It was never used once.

What came back

MEASURED ACROSS 118 LABELLED DECISIONS	RESULT
Share taken by the two easiest labels	75 to 100%
Share taken by the strongest "this is wrong" labels	0 to 2%
Times one label fired across 22 chances	0
Times two more fired across 96 chances	0

The model was not lying and it was not broken. Every label it chose was defensible. It kept choosing the ones that were easiest to defend.

The part that surprised me

Tightening one label did not push the model toward the correct label. It pushed the model to the next label whose test was easier to satisfy.

I tried the obvious fix: a strict written test for each label, so you could not use one without showing your work. Use of the loosest label dropped from seven of eight cases to one of eight, which looked like a win.

The traffic did not move to the label I was trying to promote. That one went from one case to two. A different label, one I had not touched, went from zero cases to four. The fix relocated the problem rather than removing it.

I changed three things in that round, so the shift is measured but the cause is not isolated.

A second thing worth knowing

One system splits a paragraph into separate claims before labelling each one. I ran the same paragraph through it three times. It produced 14 claims, then 8, then 10.

Any design that stores labelled claims to compare them later needs those claims to line up between runs. When the same paragraph splits 14 ways one time and 8 the next, there is nothing stable to line up.

The ten-minute check

1. Take whatever labelling scheme you already use.
2. Give it something that states its own answer, so the "nothing here" label is the correct one.
3. Count how often it says so.

That is the whole experiment. It tells you whether your labels are controls or decoration.

Where pin-check comes in

The obvious fix for a label nobody uses is to stop asking for a judgement and ask for a computation instead. A label that fires on something a script can decide does not get talked out of firing.

pin-check is that, for one narrow question: does every precise number in a document still trace to the evidence it came from? It reports each value as traced, rounded, or currently undefined, and it

decides by hashing files and comparing bytes rather than by judging. Standard library, no install, no protocol to adopt.

It found a factual error in my own writing within two minutes of existing, which is the only endorsement worth giving it.

118 labelled decisions across four experiments and two labelling systems, one model family. Full record with per-experiment counts, the predictions written before each run, and four ways to check the result: github.com/JakeTOpenSource/Resilience-Ledger · [research/label-absorption-note](#)